Geometry-Aware Prototype-Conditioned Learning in Feed Forward Neural Networks

NIVHSA
India

March 26, 2026

Abstract

We propose a geometry-aware learning framework for feed forward neural networks in which local structure in hidden representation space modulates the effective parameters used for prediction. Instead of applying a single globally shared output mapping to every sample, we associate the hidden representation with a small set of latent prototypes, compute soft prototype memberships, and use these memberships to form an input-dependent mixture of output experts. We further introduce an auxiliary local target in hidden space, defined by the same memberships, so that the hidden representation is regularized by local geometry. The resulting model is trained end-to-end by gradient descent on a differentiable objective consisting of the supervised output loss and the auxiliary local loss. The method is motivated by the hypothesis that destructive interference between distant data regions can be reduced when learning is informed by local geometry in representation space.

1 Introduction

In a standard feed forward neural network (FFNN), the same parameter tensors are shared across all samples, and training is typically driven by a global output loss propagated through the entire network. This paradigm has been highly successful, but it also couples the updates induced by all samples through a common parameter set. When two samples arise from distinct regions of the data distribution, their induced updates may interfere.

The central idea of this work is that neural learning should depend not only on the global output error, but also on the *local geometry* of the hidden representation space. We therefore augment a one-hidden-layer FFNN with three ingredients:

1. latent prototypes in hidden space,
2. soft memberships of each sample to those prototypes,
3. and a prototype-conditioned auxiliary target in hidden space.

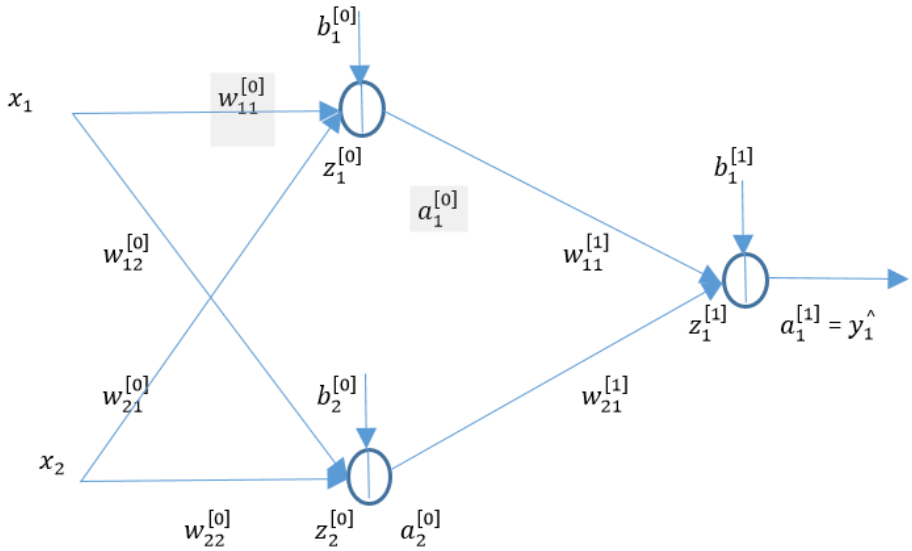

Figure 1: Backbone one-hidden-layer feed forward neural network. In the proposed method, the hidden representation is further coupled to latent prototypes, soft prototype memberships, a prototype-conditioned mixture of output experts, and an auxiliary local loss. The figure should therefore be interpreted as the base architecture on top of which the geometry-aware mechanism is added.

The memberships determine how strongly each sample uses a collection of output experts, thereby producing input-dependent effective parameters. The same memberships also define a local hidden-space target, which regularizes the hidden representation.

For clarity and consistency with the implementation, the model studied in this paper is a one-hidden-layer FFNN trained end-to-end by automatic differentiation. The theoretical analysis later in the paper is presented as a simplified mechanism analysis that isolates the interference-reduction effect of prototype-conditioned gating; it is not claimed to be a full convergence analysis of the exact nonlinear optimization dynamics.

2 Geometry-aware prototype-conditioned model

2.1 Backbone hidden representation

Let $\mathbf{x} \in \mathbb{R}^d$ be the input. We consider a one-hidden-layer FFNN with hidden dimension H. The hidden representation is

$$\mathbf{h}(\mathbf{x}) = \tanh(W_h \mathbf{x} + \mathbf{b}_h), \tag{1}$$

where $W_h \in \mathbb{R}^{H\times d}$ and $\mathbf{b}_h \in \mathbb{R}^H$ are trainable parameters.

For a C-class classification problem, the network ultimately outputs logits in $\mathbb{R}^C$ and probabilities

$$\mathbf{p}(\mathbf{x}) = \mathrm{softmax}(\mathbf{z}(\mathbf{x})). \tag{2}$$

2.2 Latent prototypes and soft memberships

We introduce K trainable latent prototypes

$$\mathbf{c}_1, \mathbf{c}_2, \ldots, \mathbf{c}_K \in \mathbb{R}^H$$

in hidden representation space. For an input $\mathbf{x}$, the hidden representation $\mathbf{h}(\mathbf{x})$ is softly assigned to these prototypes via

$$g_k(\mathbf{x}) = \frac{\exp\left(-\frac{\|\mathbf{h}(\mathbf{x})-\mathbf{c}_k\|_2^2}{\tau}\right)}{\sum_{r=1}^{K}\exp\left(-\frac{\|\mathbf{h}(\mathbf{x})-\mathbf{c}_r\|_2^2}{\tau}\right)}, \qquad k = 1, \ldots, K, \tag{3}$$

where $\tau > 0$ is a temperature parameter.

The membership vector

$$\mathbf{g}(\mathbf{x}) = \big(g_1(\mathbf{x}), \ldots, g_K(\mathbf{x})\big)$$

encodes the local geometry of $\mathbf{x}$ in hidden space.

2.3 Prototype-conditioned output experts

Instead of one global output map, we use K output experts:

$$W_1^{\mathrm{exp}}, \ldots, W_K^{\mathrm{exp}} \in \mathbb{R}^{C\times H}, \qquad \mathbf{b}_1^{\mathrm{exp}}, \ldots, \mathbf{b}_K^{\mathrm{exp}} \in \mathbb{R}^C.$$

Each expert produces logits

$$\mathbf{z}_k^{\mathrm{exp}}(\mathbf{x}) = W_k^{\mathrm{exp}}\,\mathbf{h}(\mathbf{x}) + \mathbf{b}_k^{\mathrm{exp}}. \tag{4}$$

The final logits are a membership-weighted mixture of these experts:

$$\mathbf{z}(\mathbf{x}) = \sum_{k=1}^{K} g_k(\mathbf{x})\,\mathbf{z}_k^{\mathrm{exp}}(\mathbf{x}). \tag{5}$$

Equation (5) is the implementation-consistent form of an input-dependent effective output layer. Different regions of hidden space therefore induce different effective parameter usage.

2.4 Geometry-aware local target and auxiliary loss

To make learning explicitly sensitive to local geometry in hidden space, we introduce trainable prototype-associated target vectors

$$\boldsymbol{\phi}_1, \ldots, \boldsymbol{\phi}_K \in \mathbb{R}^H.$$

For a given input $\mathbf{x}$, the auxiliary target is defined by

$$\mathbf{t}(\mathbf{x}) = \sum_{k=1}^{K} g_k(\mathbf{x})\, \boldsymbol{\phi}_k. \tag{6}$$

The auxiliary local loss is then

$$\mathcal{L}_{\text{loc}}(\mathbf{x}) = \frac{1}{2} \left\| \mathbf{h}(\mathbf{x}) - \mathbf{t}(\mathbf{x}) \right\|_2^2. \tag{7}$$

Thus, the same local geometry information that determines the effective output map also determines a prototype-conditioned regularization target for the hidden representation.

2.5 Supervised loss and end-to-end objective

For classification, let $\mathbf{y} \in \{0,1\}^C$ denote the one-hot label vector. The supervised output loss is

$$\mathcal{L}_{\text{out}}(\mathbf{x}, \mathbf{y}) = -\sum_{c=1}^{C} y_c \log p_c(\mathbf{x}), \tag{8}$$

where $\mathbf{p}(\mathbf{x})$ is defined in (2).

The implementation-consistent training objective for one sample is

$$\mathcal{L}_{\text{impl}}(\mathbf{x}, \mathbf{y}) = \mathcal{L}_{\text{out}}(\mathbf{x}, \mathbf{y}) + \lambda\, \mathcal{L}_{\text{loc}}(\mathbf{x}), \tag{9}$$

where $\lambda \geq 0$ controls the strength of the geometry-aware local regularization.

Over a training set $\mathcal{D} = \{(\mathbf{x}^{(n)}, \mathbf{y}^{(n)})\}_{n=1}^{N}$, the empirical objective is

$$\mathcal{J}(\Theta) = \frac{1}{N} \sum_{n=1}^{N} \mathcal{L}_{\text{impl}}(\mathbf{x}^{(n)}, \mathbf{y}^{(n)}), \tag{10}$$

where

$$\Theta = \left\{ W_h, \mathbf{b}_h, \{W_k^{\text{exp}}, \mathbf{b}_k^{\text{exp}}\}_{k=1}^{K}, \{\mathbf{c}_k\}_{k=1}^{K}, \{\boldsymbol{\phi}_k\}_{k=1}^{K} \right\}. \tag{11}$$

2.6 Training rule

All trainable quantities in (11) are optimized jointly by gradient descent on the differentiable objective (10). For a minibatch estimate of the loss, the parameter update is

$$\Theta \leftarrow \Theta - \eta\, \nabla_{\Theta} \mathcal{J}(\Theta), \tag{12}$$

where $\eta > 0$ is the learning rate.

This is important: in the implementation used in experiments, the memberships, prototypes, expert parameters, and auxiliary target vectors are all part of a single computational graph. Hence gradients are allowed to flow through the membership function in (3), through the expert mixture in (5), and through the auxiliary target in (6). In this sense, the implemented model is an end-to-end differentiable geometry-aware architecture rather than a manually decoupled layer-wise update scheme.

2.7 Interpretation in terms of local geometry

The role of local geometry enters the model in two coupled ways.

First, the prototype memberships modulate the effective output computation via (5). Samples that lie in different regions of hidden space therefore activate different mixtures of output experts.

Second, the same memberships define the auxiliary target (6), which regularizes the hidden representation toward a geometry-dependent direction. Therefore, the hidden representation is shaped both by the supervised label signal and by the local geometric structure induced by the prototypes.

This is the sense in which the present method uses local geometry information: local distances in hidden representation space influence both the forward computation and the learned hidden-space regularization.

3 Illustration for the one-hidden-layer network

For the one-hidden-layer FFNN in Fig. 1, the hidden representation is

$$\mathbf{h}(\mathbf{x}) = \tanh(W_h \mathbf{x} + \mathbf{b}_h).$$

Given prototypes $\{\mathbf{c}_k\}_{k=1}^{K}$, the memberships are computed by (3). The output logits of the kth expert are

$$\mathbf{z}_k^{\exp}(\mathbf{x}) = W_k^{\exp} \mathbf{h}(\mathbf{x}) + \mathbf{b}_k^{\exp},$$

and the final logits are

$$\mathbf{z}(\mathbf{x}) = \sum_{k=1}^{K} g_k(\mathbf{x})\, \mathbf{z}_k^{\exp}(\mathbf{x}).$$

Thus, the output mapping is no longer globally fixed across all inputs; it depends on the sample's location in hidden representation space. The same memberships define the auxiliary target

$$\mathbf{t}(\mathbf{x}) = \sum_{k=1}^{K} g_k(\mathbf{x})\, \boldsymbol{\phi}_k,$$

which induces the local loss in (7). The model is then trained end-to-end by minimizing (10).

Algorithm 1 Geometry-Aware Prototype-Conditioned End-to-End Training

Require: Training set $\mathcal{D} = \{(\mathbf{x}^{(n)}, \mathbf{y}^{(n)})\}_{n=1}^{N}$, number of prototypes K, learning rate η, temperature τ, local-loss weight λ

```
Initialize hidden-layer parameters W_h, b_h
Initialize prototypes {c_k}_{k=1}^K, local target vectors {φ_k}_{k=1}^K
Initialize expert parameters {W_k^exp, b_k^exp}_{k=1}^K
for each epoch do
    for each minibatch B ⊂ D do
        for each sample (x, y) ∈ B do
            Compute hidden representation h(x) = tanh(W_h x + b_h)
            Compute memberships g_k(x) using (3)
            Compute expert logits z_k^exp(x) using (4)
            Compute mixed logits z(x) = Σ_{k=1}^K g_k(x) z_k^exp(x)
            Compute local target t(x) = Σ_{k=1}^K g_k(x) φ_k
            Compute supervised loss L_out(x, y)
            Compute local loss L_loc(x)
            Form total loss L_impl(x, y) = L_out(x, y) + λ L_loc(x)
        end for
        Compute minibatch average of L_impl
        Backpropagate through the full computational graph
        Update all parameters in Θ using one optimizer step
    end for
end for
```

4 Pseudo-code

5 Discussion

The proposed formulation differs from earlier hand-designed local update proposals in an important way: the present implementation is fully differentiable and internally closed. The prototype memberships, expert parameters, prototypes themselves, and auxiliary target vectors are all learned jointly by gradient descent on the implemented objective. Consequently, there is no ambiguity about how the prototype vectors or local target vectors are updated, and there is no mismatch between the stated objective and the implemented optimizer.

At the same time, the model preserves the central geometry-aware idea: samples in different regions of hidden space induce different effective mixtures of output experts, and the same local geometry defines the auxiliary target used to regularize the hidden representation.

The intended benefit is not necessarily a universal improvement over standard backpropagation, but rather a geometry-aware inductive bias that may reduce destructive interference between distant data regions and improve performance in settings where globally shared updates are suboptimal.

6 A simplified interference-reduction result

The implementation in this paper is trained end-to-end by gradient descent on the differentiable objective (10). The result below should therefore be interpreted as a simplified mechanism analysis that isolates the interference-reduction effect of geometry-aware gating. It is not claimed to be a full convergence theorem for the exact nonlinear optimizer used in the implementation.

6.1 Setup

Consider a one-hidden-layer network with hidden representation $\mathbf{h}(\mathbf{x}) \in \mathbb{R}^d$. For theoretical analysis, suppose the output layer is written as a collection of region-specific matrices

$$W_1, \ldots, W_K \in \mathbb{R}^{m \times d},$$

and define the effective output-layer weight

$$\widetilde{W}(\mathbf{x}) = \sum_{k=1}^{K} g_k(\mathbf{x})\, W_k, \tag{13}$$

where $g_k(\mathbf{x}) \geq 0$ and $\sum_{k=1}^{K} g_k(\mathbf{x}) = 1$.

To isolate the effect of gating, consider the simplified sample-induced update

$$\Delta W_k(\mathbf{x}) = -\eta\, g_k(\mathbf{x})\, \boldsymbol{\delta}(\mathbf{x})\, \mathbf{h}(\mathbf{x})^\top, \tag{14}$$

where $\eta > 0$ is the learning rate and $\boldsymbol{\delta}(\mathbf{x}) \in \mathbb{R}^m$ is a generic error vector. This update is not the exact optimizer of the full implemented model; rather, it captures the geometry-gated rank-one update component that the proposition studies.

To measure interaction between two sample-induced updates, define the total overlap

$$\Omega(\mathbf{x}_p, \mathbf{x}_q) = \sum_{k=1}^{K} \langle \Delta W_k(\mathbf{x}_p),\, \Delta W_k(\mathbf{x}_q) \rangle_F\,, \tag{15}$$

where $\langle A, B \rangle_F = \operatorname{tr}(A^\top B)$ denotes the Frobenius inner product.

6.2 Preliminary lemma

Lemma 6.1. *Let* $\mathbf{u}, \mathbf{v} \in \mathbb{R}^m$ *and* $\mathbf{r}, \mathbf{s} \in \mathbb{R}^d$. *Then*

$$\left\langle \mathbf{u}\mathbf{r}^\top, \mathbf{v}\mathbf{s}^\top \right\rangle_F = (\mathbf{u}^\top \mathbf{v})(\mathbf{r}^\top \mathbf{s}). \tag{16}$$

Proof. Using the definition of the Frobenius inner product,

$$\left\langle \mathbf{u}\mathbf{r}^\top, \mathbf{v}\mathbf{s}^\top \right\rangle_F = \operatorname{tr}\big((\mathbf{u}\mathbf{r}^\top)^\top (\mathbf{v}\mathbf{s}^\top)\big) = \operatorname{tr}\big(\mathbf{r}\mathbf{u}^\top \mathbf{v}\mathbf{s}^\top\big)\,.$$

Since $\mathbf{u}^\top \mathbf{v}$ is a scalar,

$$\operatorname{tr}\big(\mathbf{r}\mathbf{u}^\top \mathbf{v}\mathbf{s}^\top\big) = (\mathbf{u}^\top \mathbf{v})\operatorname{tr}(\mathbf{r}\mathbf{s}^\top) = (\mathbf{u}^\top \mathbf{v})(\mathbf{r}^\top \mathbf{s}),$$

which proves the claim. □

6.3 Main proposition

Proposition 6.2 (Reduction of cross-region update overlap)**.** *Consider two samples* $\mathbf{x}_p$ *and* $\mathbf{x}_q$ *with updates defined by* (14)*. Then their total overlap satisfies*

$$\Omega(\mathbf{x}_p, \mathbf{x}_q) = \eta^2 \Big(\boldsymbol{\delta}(\mathbf{x}_p)^\top \boldsymbol{\delta}(\mathbf{x}_q)\Big)\Big(\mathbf{h}(\mathbf{x}_p)^\top \mathbf{h}(\mathbf{x}_q)\Big) \sum_{k=1}^{K} g_k(\mathbf{x}_p)\, g_k(\mathbf{x}_q). \tag{17}$$

Hence,

$$|\Omega(\mathbf{x}_p, \mathbf{x}_q)| \le \eta^2 \|\boldsymbol{\delta}(\mathbf{x}_p)\|_2 \|\boldsymbol{\delta}(\mathbf{x}_q)\|_2 \|\mathbf{h}(\mathbf{x}_p)\|_2 \|\mathbf{h}(\mathbf{x}_q)\|_2 \sum_{k=1}^{K} g_k(\mathbf{x}_p)\, g_k(\mathbf{x}_q). \tag{18}$$

In particular, if the two samples have concentrated memberships on different latent prototypes, then the quantity

$$\sum_{k=1}^{K} g_k(\mathbf{x}_p)\, g_k(\mathbf{x}_q)$$

is small, and therefore the overlap between their updates is small.

Proof. From (14),

$$\Delta W_k(\mathbf{x}_p) = -\eta\, g_k(\mathbf{x}_p)\, \boldsymbol{\delta}(\mathbf{x}_p)\, \mathbf{h}(\mathbf{x}_p)^\top$$

and

$$\Delta W_k(\mathbf{x}_q) = -\eta\, g_k(\mathbf{x}_q)\, \boldsymbol{\delta}(\mathbf{x}_q)\, \mathbf{h}(\mathbf{x}_q)^\top.$$

Therefore,

$$\langle \Delta W_k(\mathbf{x}_p), \Delta W_k(\mathbf{x}_q) \rangle_F = \eta^2 g_k(\mathbf{x}_p) g_k(\mathbf{x}_q) \left\langle \boldsymbol{\delta}(\mathbf{x}_p)\mathbf{h}(\mathbf{x}_p)^\top,\, \boldsymbol{\delta}(\mathbf{x}_q)\mathbf{h}(\mathbf{x}_q)^\top \right\rangle_F.$$

Applying Lemma 6.1,

$$\left\langle \boldsymbol{\delta}(\mathbf{x}_p)\mathbf{h}(\mathbf{x}_p)^\top,\, \boldsymbol{\delta}(\mathbf{x}_q)\mathbf{h}(\mathbf{x}_q)^\top \right\rangle_F = \Big(\boldsymbol{\delta}(\mathbf{x}_p)^\top \boldsymbol{\delta}(\mathbf{x}_q)\Big)\Big(\mathbf{h}(\mathbf{x}_p)^\top \mathbf{h}(\mathbf{x}_q)\Big).$$

Summing over k gives

$$\Omega(\mathbf{x}_p, \mathbf{x}_q) = \eta^2 \Big(\boldsymbol{\delta}(\mathbf{x}_p)^\top \boldsymbol{\delta}(\mathbf{x}_q)\Big)\Big(\mathbf{h}(\mathbf{x}_p)^\top \mathbf{h}(\mathbf{x}_q)\Big) \sum_{k=1}^{K} g_k(\mathbf{x}_p)\, g_k(\mathbf{x}_q),$$

which proves (17).

The bound (18) follows directly from Cauchy–Schwarz:

$$|\boldsymbol{\delta}(\mathbf{x}_p)^\top \boldsymbol{\delta}(\mathbf{x}_q)| \le \|\boldsymbol{\delta}(\mathbf{x}_p)\|_2 \|\boldsymbol{\delta}(\mathbf{x}_q)\|_2,$$

$$|\mathbf{h}(\mathbf{x}_p)^\top \mathbf{h}(\mathbf{x}_q)| \le \|\mathbf{h}(\mathbf{x}_p)\|_2 \|\mathbf{h}(\mathbf{x}_q)\|_2.$$

Substituting these inequalities into (17) yields the result. □

6.4 Interpretation

Proposition 6.2 shows that the overlap between two sample-induced updates is scaled by the membership similarity factor

$$S(\mathbf{x}_p, \mathbf{x}_q) = \sum_{k=1}^{K} g_k(\mathbf{x}_p)\, g_k(\mathbf{x}_q). \tag{19}$$

This factor lies in $[0, 1]$. It is large when the two samples activate similar latent regions and small when they activate different latent regions. Therefore, geometry-aware gating naturally reduces coupling between samples that belong to different parts of hidden representation space.

By contrast, in a globally shared output layer without prototype-conditioned gating, there is no explicit factor analogous to (19) that suppresses cross-region interaction. The proposition therefore formalizes the key qualitative mechanism behind the proposed model.

6.5 A useful corollary

Corollary 6.3. *Assume that for sample $\mathbf{x}_p$ there exists an index k_p such that $g_{k_p}(\mathbf{x}_p) \geq 1 - \varepsilon$, and for sample $\mathbf{x}_q$ there exists an index $k_q \neq k_p$ such that $g_{k_q}(\mathbf{x}_q) \geq 1 - \varepsilon$, where $\varepsilon \in [0, 1]$. Then*

$$\sum_{k=1}^{K} g_k(\mathbf{x}_p)\, g_k(\mathbf{x}_q) \leq 2\varepsilon - \varepsilon^2. \tag{20}$$

Consequently,

$$|\Omega(\mathbf{x}_p, \mathbf{x}_q)| \leq \eta^2 \|\boldsymbol{\delta}(\mathbf{x}_p)\|_2 \|\boldsymbol{\delta}(\mathbf{x}_q)\|_2 \|\mathbf{h}(\mathbf{x}_p)\|_2 \|\mathbf{h}(\mathbf{x}_q)\|_2 (2\varepsilon - \varepsilon^2). \tag{21}$$

Proof. Since $g_{k_p}(\mathbf{x}_p) \geq 1 - \varepsilon$, the total mass on all other indices is at most ε. Likewise, since $g_{k_q}(\mathbf{x}_q) \geq 1 - \varepsilon$ and $k_q \neq k_p$, the mass outside k_q is at most ε. Therefore, the dot product between the two membership vectors is maximized when the remaining mass is concentrated as much as possible, which yields

$$\sum_{k=1}^{K} g_k(\mathbf{x}_p)\, g_k(\mathbf{x}_q) \leq \varepsilon + \varepsilon(1 - \varepsilon) = 2\varepsilon - \varepsilon^2.$$

Substituting this into (18) completes the proof. □

6.6 Remark on scope

The above result does not establish global convergence of the full nonlinear model trained by (12), nor does it claim that all forms of interference are eliminated. Rather, it isolates one mathematically precise mechanism: when geometry-aware memberships concentrate on different latent regions, the overlap between the corresponding gated update components is explicitly suppressed. This provides a concrete theoretical basis for the claim that local geometry can reduce cross-region interference.

References

[1] Andina, D., Pham, D., Vega-Corona, A., Seijas, J., Torres-García, J. Neural Networks Historical Review.

[2] Rumelhart, D., Hinton, G., Williams, R. Learning representations by back-propagating errors. *Nature*, 323, 533–536, 1986.